



하네스 엔지니어링

AI 에이전트 시대의 새로운 엔지니어링 패러다임

Noah | 2026.04

AGENDA

01

패러다임의 진화

프롬프트 → 컨텍스트 → 하네스

02

하네스의 핵심

세 기둥: 제약, 피드백 루프, 상태 관리

03

실전 적용

성숙도 모델과 구축 여정

0 1

패러다임의 진화

프롬프트 → 컨텍스트 → 하네스로 이어지는 포괄적 확장

세 가지 패러다임의 진화

각 단계는 이전을 부정하지 않고 포함하며 확장한다.

2022-2024



프롬프트 엔지니어링

"어떻게 말해야 하는가?"

단일 입력 최적화
Few-shot, CoT, 역할 부여

2025



컨텍스트 엔지니어링

"모델이 무엇을 알아야 하는가?"

정보 환경 구성
RAG, MCP, 도구 정의

2026



하네스 엔지니어링

"시스템을 어떻게 설계해야 하는가?"

런타임 환경 설계
제약, 피드백 루프, 상태 관리

컴퓨터 아키텍처 비유

Application (에이전트)

실제 작업을 수행하는 AI 에이전트

Operating System (하네스)

프로세스 관리, 메모리 관리, I/O 제어

◀ 하네스 엔지니어링

RAM (컨텍스트 윈도우)

RAG, MCP, 대화 히스토리, 도구 정의

◀ 컨텍스트 엔지니어링

CPU (모델)

프롬프트 = CPU에 전달하는 명령어

◀ 프롬프트 엔지니어링

OS 없이 CPU에서 직접 프로그램을 돌리면? → 메모리 충돌, 무한 루프 보호 장치 없음

프롬프트에서 컨텍스트로: 한계의 발견

프롬프트 엔지니어링

핵심: 단일 입력을 최적화하여 최선의 단일 출력을 얻는 기술

기법:

Few-shot, Chain-of-Thought, 역할 부여, 출력 형식 지정

한계:

관련 코드를 모델이 볼 수 없음, 프로젝트 아키텍처 규칙 모름, 후속 조치 체계 없음

비유:

"완벽한 이메일 한 통을 쓰는 기술"

컨텍스트 엔지니어링

핵심: 모델에게 동적으로 구성된 컨텍스트 윈도우를 제공

기법:

RAG, MCP, 대화 히스토리 관리, 도구 정의, 예산 관리

한계:

컨텍스트 윈도우 제한, 단일 에이전트 편향, 세션 간 상태 유실, 47번째에서 규칙 망각

비유:

"모든 첨부파일을 빠짐없이 붙인 이메일"

“

"에이전트가 실수할 때마다,
다음에 더 잘하라고 말하는 게 아니라,
그 특정 실수가 구조적으로 반복되기
어렵도록 시스템을 바꾸는 것."

— Mitchell Hashimoto, Terraform 창시자 (2026.02)

프롬프트는 "요청" 이고, 하네스는 "강제" 다.

왜 지금 하네스 엔지니어링인가

세 가지 변화가 동시에 수렴했다.



모델의 상품화

Claude, GPT, Gemini가
벤치마크에서 좁은 범위로 수렴.
차이를 만드는 것은
모델을 감싸는 시스템.



에이전트의 프로덕션 진입

2026년, 에이전트가 실제
프로덕션 코드를 작성.
"다운되면 안 된다" 수준의
신뢰도 요구.



벤치마크의 한계 노출

벤치마크는 단일 턴 측정.
프로덕션 에이전트는
수백 단계를 실행.
50단계 후 방향 이탈에
1% 향상은 무의미.

프롬프트로 해결할 수 없는 문제들

문제	프롬프트/컨텍스트로는...	하네스로는...
자기 평가 편향	"비판적으로 리뷰하라" → 자기 코드를 칭찬	Generator와 Evaluator를 물리적으로 분리
컨텍스트 불안	컴팩션해도 "마무리해야 한다"는 충동	컨텍스트 리셋 + 파일 핸드오프
규칙 망각	CLAUDE.md에 써도 47번째에서 잊힘	Hook/린터로 자동 감지
아키텍처 침범	"이 레이아웃만 수정하라" → 편의상 위반	allowed_tools로 도구 레벨 접근 차단
무한 루프	"5번 이상 시도 금지" → 무시 가능	max_iterations로 하드 리밋 강제
상태 유실	세션 전환 시 이전 맥락 소실	파일 기반 핸드오프로 상태 영속화

0 2

하네스의 핵심

세 기둥: 제약, 피드백 루프, 상태 관리

하네스 엔지니어링의 세 기둥



제약 Constraints

"무엇을 하지 못하게 할 것인가"

탐색 공간을 줄여
에이전트를 집중시킨다.

CLAUDE.md 규칙 주입
allowed_tools 도구 제한
커스텀 린터/구조 테스트



피드백 루프 Feedback Loops

"실수를 어떻게 잡을 것인가"

실수를 감지하고
수정하는 순환 구조.

자기 리뷰 (약)
독립 리뷰어 (중)
자동화 검증 (강)



상태 관리 State Mgmt

"맥락을 어떻게 유지할 것인가"

컨텍스트 윈도우 밖에
상태를 저장하고 복원.

파일 핸드오프
컨텍스트 리셋/컴팩션
서브에이전트 격리

기둥 1: 제약 — 직관에 반하는 생산성

탐색 공간이 줄어들면 에이전트는 더 집중된 출력을 생성한다.

CLAUDE.md — 구조적 제약 예시

절대 금지

- @Transactional 내부에서 외부 API 호출 금지
- Redis 분산락을 트랜잭션 경계 안에서 금지
- ALTER TABLE 전 INSTANT 가능 여부 미확인 시 금지

구조적 규칙

- domain은 infrastructure를 import 불가
- API 응답 DTO와 Entity 직접 매핑 금지

핵심 인사이트

CLAUDE.md는 매 세션 시작 시 자동 로드된다. 에이전트가 "잊을 수 없는" 제약이 된다.

OpenAI Codex 팀: 엄격한 레이어 아키텍처를 린터로 강제
→ 에이전트가 레이어를 침범하는 코드 생성이 불가능해짐

Hook 시스템 예시

PreToolUse hook:
@Transactional 내부에서
RedisTemplate.setIfAbsent() 호출을
자동으로 감지하고 경고.

→ 모델이 규칙을 "잊어도" 메커니즘이 잡아낸다.

기둥 2: 피드백 루프 — 실수를 잡는 순환 구조

Anthropic 인사이트: 에이전트에게 자기 작업을 평가하라고 하면, 미흡한 결과에도 자신있게 칭찬한다. 독립된 평가자를 회의적으로 튜닝하는 것이 훨씬 다루기 쉽다.

약함

자기 리뷰

같은 에이전트가 작업 후 자체 검토
→ 자기 평가 편향 존재

중간

독립 리뷰어

별도의 서브에이전트가 검토
→ 코드 수정 권한 없이 관찰만

강함

자동화된 검증

린터, 테스트, 타입체크 등
→ 거짓 통과 불가 (결정적 도구)

가장 효과적인 하네스는 세 가지를 계층적으로 조합한다: 자동화 검증 → 독립 리뷰어 → 인간 체크포인트

기둥 3: 상태 관리 — 유한한 기억을 넘어서

전략	동작	장점	단점
컴팩션	이전 대화를 요약하여 축소	연속성 유지	컨텍스트 불안 가능
컨텍스트 리셋	세션을 완전히 새로 시작	깨끗한 슬레이트	핸드오프 필요
파일 핸드오프	파일에 상태를 기록, 다음 세션에서 읽기	영속적, 인간도 읽기 가능	파일 구조 설계 필요

Hashimoto의 핵심 실천

progress.txt 파일에 현재까지의 진행 상황과 다음 단계를 기록하고, 매 세션 시작 시 에이전트가 이 파일을 읽도록 한다. 인간이 TODO 리스트를 사용하는 것과 동일한 원리.

파일 핸드오프 구조: plan.md → sprint-contract.md → sprint-result.md → validation-result.md → lessons-learned.md

세 패러다임은 포함 관계다

하네스 엔지니어링

컨텍스트 엔지니어링

프롬프트 엔지니어링

Few-shot, CoT
역할 부여, 출력 형식

제약 시스템

allowed_tools, 린터, 구조 테스트

피드백 루프

Generator-Evaluator 분리, 2단계 리뷰

상태 관리

파일 핸드오프, 서브에이전트 격리

0 3

실전 적용

오늘부터 시작하는 하네스 구축

실전 비교: 같은 코드 리뷰, 세 가지 접근

Redis 분산락 + DB 트랜잭션 데드락 시나리오

프롬프트

"시니어 개발자로서
이 코드의 thread safety
이슈를 리뷰하라"

→ 단일 턴, 맥락 없음
→ 관련 코드를 볼 수 없음

= 완벽한 이메일 한 통

컨텍스트

CLAUDE.md + 관련 파일
+ 과거 장애 기록 주입

→ 프로젝트 맥락 안에서 판단
→ 장시간 세션에서 규칙 잊힘
→ 단일 에이전트 편향

= 첨부파일 완벽한 이메일

하네스

CLAUDE.md: 구조적 제약
Hook: 패턴 자동 감지
역할 분리: Reviewer 독립
피드백: 반복 → 규칙 승격
상태: 파일 핸드오프

= 자동화된 품질 시스템

클린 아키텍처로 이해하는 하네스

원칙은 불변이고, 구현만 교체 가능하다 — 의존성 역전 원칙

도메인 레이어 (원칙)

"역할을 분리하라" / "평가자를 독립시켜라"
"컨텍스트를 격리하라" / "제약이 에이전트를 집중시킨다"

Port (인터페이스)

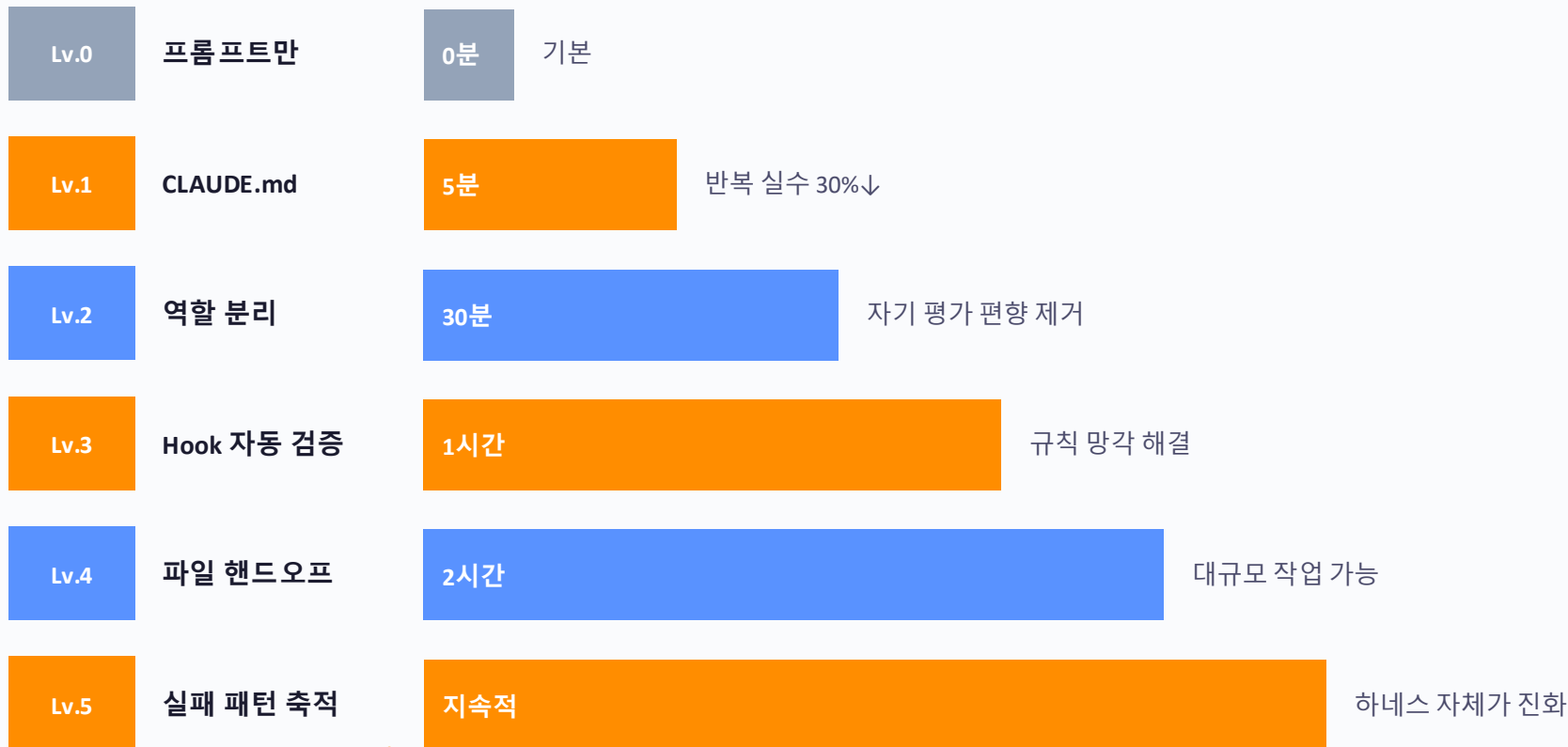
"계획과 실행을 분리하는 인터페이스"
"생성자와 평가자를 분리하는 인터페이스"

Adapter (구현체)

Plan Mode · Superpowers · .claude/agents/
Agent SDK · Ralph Loop · CLAUDE.md · Hook 시스템

같은 하네스 원칙을 다른 어댑터로 구현할 수 있다.

하네스 성숙도 모델: 5분부터 시작하라



핵심: 실수할 때마다 "이번만 고치는" 것이 아니라 "시스템에 규칙을 추가하는" 습관이 본질이다.

Hashimoto의 6단계 여정: 챗봇에서 하네스까지

1

챗봇 버리기

프롬프트만으로는 부족

2

자기 작업 재현

에이전트의 실패 패턴 학습

3

퇴근 전 에이전트

자율 실행의 가치 발견

4

확실한 작업 위임

작업 분류 능력이 핵심

5

하네스 엔지니어링

일회성 수정이 아닌 구조적 개선

6

항상 에이전트 실행

하네스가 충분히 성숙하면 가능

하네스는 진화한다

Anthropic 메타 원칙: "하네스의 모든 컴포넌트는 '모델이 스스로 할 수 없는 것'에 대한 가정을 인코딩한다. 그 가정들은 스트레스 테스트할 가치가 있다 — 잘못된 가정일 수 있고, 모델이 개선되면서 빠르게 진부해질 수 있기 때문이다."

분기마다 스스로에게 질문:

- 이 제약은 아직 필요한가? 모델이 이미 이걸 잘 하지 않나?
- 이 피드백 루프가 잡는 문제가 아직 발생하는가?
- 새 모델 버전으로 가능해진 더 나은 패턴은 없는가?



"There is no silver bullet."

— Fred Brooks, 1986

하네스 엔지니어링을 구현하는 정해진 완벽한 방법은 없다.

모델이 진화하면, 하네스도 함께 진화해야 한다.

오늘 시작하고, 실패에서 배우고, 시스템을 계속 발전시켜라.

Q & A

감사합니다